

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	P Likith Reddy
Reg. No	192425134

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

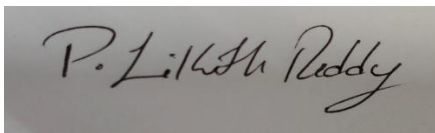
Duration: 30 minutes

Marks: 25

Code of Conduct:

I P Likith Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



P Likith Reddy

Signature of the student:

Name of the student:

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

When training a machine learning model with gradient descent on a convex loss function, using a high initial learning rate followed by gradual decay is a common optimization strategy. It aims to combine fast initial progress with stable convergence near the optimum.

Effect on Convergence

A high initial learning rate causes large parameter updates, allowing the algorithm to:

- Move rapidly toward the region containing the global minimum.
- Reduce the loss significantly in the early stages of training.
- Spend less time making tiny improvements when far from the optimum.

Possible Risks

Although effective, this strategy has several risks.

- Learning rate too high initially
 - Updates may overshoot the minimum.
 - The loss may oscillate or even increase.
 - In extreme cases, the optimization may diverge instead of converging.
- Decay too quickly
 - The learning rate becomes very small before reaching the optimum.
 - Learning slows dramatically, leading to premature convergence and inefficient optimization.
- Decay too slowly
 - The algorithm continues making large updates near the minimum.
 - Oscillations persist, preventing accurate convergence.
- Sensitive Hyperparameter Selection
 - Choosing the initial learning rate and decay schedule requires careful tuning.
 - Poor choices can make performance worse than a well-selected constant learning rate.

Justification

The intuition behind this strategy is that optimization has two distinct phases:

- Far from the optimum: Large steps are desirable because the gradient provides a reliable direction toward the minimum, making rapid progress possible.

- Near the optimum: Large steps become inefficient because they repeatedly overshoot the minimum. Smaller steps allow the algorithm to settle accurately into the global optimum.

A constant learning rate cannot simultaneously provide both fast early progress and precise final convergence. Therefore, a high initial learning rate with gradual decay often achieves a better balance between speed and accuracy.

Conclusion

Using a high initial learning rate followed by gradual decay generally improves optimization by combining rapid early convergence with stable final convergence. It can outperform a constant learning rate when the initial parameters are far from the optimum, the decay schedule is well designed, and the optimization problem benefits from different step sizes during different training stages. However, if the initial learning rate is excessively high or the decay schedule is poorly chosen, the algorithm may oscillate, converge slowly, or even diverge. Thus, the success of this strategy depends on selecting an appropriate initial learning rate and decay schedule.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer:

Maximum Likelihood Estimator (MLE) for the Mean μ

Assume the dataset consists of independent observations:

$$x_1, x_2, \dots, x_n \sim N(\mu, \sigma^2)$$

where:

- Mean μ is **unknown**
- Variance σ^2 is **known**

Step 1: Likelihood Function

The probability density function of a Gaussian distribution is

$$f(x_i | \mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

The likelihood of all observations is

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

Step 2: Log-Likelihood

Taking the natural logarithm,

$$\ell(\mu) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3: Differentiate

Differentiate with respect to μ :

$$\frac{d\ell}{d\mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu)$$

Set the derivative equal to zero:

$$\sum_{i=1}^n (x_i - \mu) = 0$$

$$\sum_{i=1}^n x_i = n\mu$$

Therefore,

Original amounts

Equal shares

2

4

7

$$\bar{x} = \frac{2 + 4 + 7}{3} = \frac{13}{3} = 4 \frac{1}{3}$$

Remove valueAdd value

Change two to five whole-number amounts. The picture redistributes their total into equal shares and shows the arithmetic mean.

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i = \bar{x}$$

Thus, **the MLE of the mean is simply the sample mean.**

Example

Original data:

2, 3, 4, 5, 6

Mean

$$\bar{x} = 4$$

Replace 6 with 100:

2, 3, 4, 5, 100

New mean

$$\bar{x} = 22.8$$

Although only one value changed, the estimate shifted dramatically.

Implication:

- MLE (sample mean) is **highly sensitive to outliers**.
- Even a single extreme observation can considerably distort the estimate.

Robustness of MLE

The robustness of an estimator refers to how resistant it is to abnormal or extreme observations.

Advantages

- Efficient for normally distributed data.
- Unbiased estimator:

$$E[\hat{\mu}] = \mu$$

- Consistent as sample size increases.
- Minimum variance among unbiased estimators for Gaussian data.

Limitations

- Very sensitive to outliers because it minimizes squared errors.
- Extreme observations have a disproportionately large influence on the estimate.
- Performance deteriorates if the Gaussian assumption is violated.

Hence, although MLE is statistically optimal under the normal distribution assumption, it is **not a robust estimator**.

Conclusion

For a Gaussian distribution with known variance, the Maximum Likelihood Estimator of the mean is

$$\hat{\mu}_{MLE} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Building a Machine Learning Algorithm (Model Selection)

Challenges

The dataset contains **10,000 features** but only **500 training samples** ($p \gg n$), creating a **high-dimensional, low-sample-size** problem. The main challenges are:

1. **Curse of Dimensionality**
 - The feature space becomes extremely sparse.
 - Learning meaningful decision boundaries becomes difficult.
2. **Overfitting**
 - The model may memorize the training data instead of learning general patterns.
 - High training accuracy but poor test accuracy.
3. **High Computational Cost**
 - Training with 10,000 features requires more memory and computation.
4. **Irrelevant and Redundant Features**
 - Many features may contain little or no useful information.
 - Redundant features increase model complexity.
5. **Multicollinearity**
 - Highly correlated features make parameter estimation unstable.

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Remove duplicate records.
- Normalize or standardize features.

Step 2: Feature Selection

Use statistical methods such as:

- Chi-Square Test (categorical features)
- ANOVA F-test
- Mutual Information
- L1 (LASSO) feature selection

Keep only the most informative features (e.g., reduce from 10,000 to 500–1,000 features).

Step 3: Dimensionality Reduction

Apply **Principal Component Analysis (PCA)** to further reduce dimensionality.

Example:

- 10,000 features
- ↓ Feature Selection
- 800 features
- ↓ PCA
- 100 principal components

Step 4: Regularization

Train a regularized classifier such as:

- Logistic Regression + L1/L2 Regularization
- Linear SVM
- Elastic Net

Regularization controls model complexity and improves generalization.

Step 5: Cross-Validation

Use **k-fold cross-validation** (e.g., 5-fold or 10-fold).

Justification:

- Makes efficient use of the limited 500 samples.
- Provides a more reliable estimate of model performance.
- Helps select the best hyperparameters.

Step 6: Model Evaluation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC (especially for imbalanced datasets)

Choose the model with the best validation performance rather than the highest training accuracy.

How This Pipeline Mitigates Overfitting

Step	How It Reduces Overfitting
Feature Selection	Removes irrelevant and noisy features, reducing model complexity.
PCA	Compresses correlated features into fewer informative components.
Regularization	Penalizes overly complex models and discourages large coefficients.
Cross-Validation	Detects overfitting by evaluating performance on unseen folds.

Step

Simpler Classifier (e.g., Logistic Regression/SVM)

How It Reduces Overfitting

Lower model complexity compared to deep neural networks, making it suitable for small datasets.

Conclusion

A dataset with 10,000 features and only 500 training samples is highly prone to overfitting due to the curse of dimensionality. An effective approach is to build a pipeline consisting of data preprocessing, feature selection, dimensionality reduction (PCA), regularized classification (L1/L2 or Elastic Net), and k-fold cross-validation. This strategy reduces noise, lowers model complexity, improves computational efficiency, and enhances the model's ability to generalize to unseen data, resulting in better predictive performance.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Neural Networks (Multilayer Perceptron - MLP)

Representational Power of Hidden Neurons

A **Multilayer Perceptron (MLP)** consists of an input layer, one or more hidden layers, and an output layer. Each hidden neuron computes:

$$h_j = \sigma \left(\sum_{i=1}^n w_{ij} x_i + b_j \right)$$

where:

- x_i = input features
- w_{ij} = weights
- b_j = bias
- $\sigma(z) = \frac{1}{1+e^{-z}}$ = sigmoid activation function

The network output is:

$$y = \sum_{j=1}^m v_j h_j + c$$

where m is the number of hidden neurons.

Why More Hidden Neurons Do Not Always Improve Performance

Although more hidden neurons increase model capacity, they do **not** always improve prediction accuracy on unseen data.

1. Overfitting

If the network has too many hidden neurons relative to the amount of training data:

- It may memorize the training samples instead of learning general patterns.
- Training accuracy becomes very high.
- Test accuracy decreases because the model fails to generalize.

2. Poor Generalization

Generalization is the ability of a model to perform well on unseen data.

A network with excessive hidden neurons often:

- Learns random fluctuations in the training data.
- Produces unstable predictions for new inputs.
- Has high variance.

Thus, increased complexity can reduce generalization performance.

3. Increased Computational Cost

More hidden neurons result in:

- More trainable parameters (weights and biases).
 - Longer training time.
 - Higher memory usage.
 - Greater computational requirements.
-

4. Vanishing Gradient Problem

Since the sigmoid activation function outputs values between 0 and 1, its gradients become very small for large positive or negative inputs.

As the network grows, gradients can diminish during backpropagation, leading to:

- Slow learning.
 - Difficulty updating weights effectively.
 - Reduced training efficiency.
-

Relationship Between Model Complexity and Performance

- **Too few hidden neurons (Underfitting):**
 - Model is too simple.
 - Cannot capture important patterns.
 - High training and testing error.
-

Preventing Overfitting

To improve generalization while using larger networks:

- Use **regularization** (L1/L2 penalties).
 - Apply **dropout** to randomly deactivate neurons during training.
 - Use **early stopping** to halt training before overfitting occurs.
 - Increase the size and diversity of the training dataset.
 - Perform **cross-validation** to select an appropriate number of hidden neurons.
-

Conclusion

Adding more hidden neurons in an MLP with sigmoid activation increases its **representational power** because each neuron learns additional nonlinear features, allowing the network to approximate more complex functions. However, increased capacity does not always improve performance. Excessive hidden neurons can lead to **overfitting**, poor **generalization**, increased computational cost, and training difficulties such as the **vanishing gradient problem**. Therefore, the best-performing model is one that balances network complexity with the available training data, achieving strong generalization rather than simply maximizing training accuracy.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Backpropagation, SGD & Curse of Dimensionality

When a deep neural network is trained using **Stochastic Gradient Descent (SGD)** on a very high-dimensional dataset, the **curse of dimensionality** affects training efficiency and model performance.

1. Impact on Gradient Updates (Backpropagation)

- In high-dimensional data, many features are irrelevant or noisy.
- Gradients become noisy and less informative, making weight updates unstable.
- This can lead to slow or inefficient learning during backpropagation.

2. Impact on Convergence Speed of SGD

- SGD requires more iterations to find the optimal solution because the optimization space is much larger.
- Gradient directions become less accurate, causing slower convergence and oscillations during training.

3. Impact on Model Generalization

- High-dimensional models contain many parameters, increasing the risk of **overfitting**.
- The network may memorize the training data instead of learning general patterns, resulting in poor performance on unseen data.

Techniques to Overcome These Issues

1. Dimensionality Reduction (e.g., PCA)

- Reduces the number of input features while preserving important information.
- Simplifies the optimization problem, leading to faster SGD convergence and reduced overfitting.

2. Regularization (L1/L2 or Dropout)

- **L1/L2 Regularization** penalizes large weights, reducing model complexity.
- **Dropout** randomly disables neurons during training, preventing the network from relying too heavily on specific features.
- Both techniques improve **generalization** and reduce overfitting.

Conclusion

The curse of dimensionality makes gradient updates noisier, slows SGD convergence, and increases the risk of overfitting. Applying **dimensionality reduction** and **regularization techniques** helps reduce model complexity, speeds up training, and improves the model's ability to generalize to unseen data.

